

MAD Batch 1

B.E / B.Tech. PRACTICAL END SEMESTER EXAMINATIONS

Sixth Semester

IT3681 - MOBILE APPLICATIONS DEVELOPMENT LABORATORY

(Regulations 2021) - SET 1 / BATCH 1 (SIMPLIFIED)

Time: 3 Hours | Max. Marks: 100

MARK ALLOCATION TABLE

AIM	ALGORITHM / UI DESIGN	PROGRAM / CODE	EXECUTION & OUTPUT	VIVA	RECORD	TOTAL
10	20	30	20	10	10	100

Ex 1 & 6: GUI Components, Fonts, and Colors

Aim: To create an Android activity with buttons to dynamically change the background color and font size of a TextView.

Algorithm:

1. Create an XML layout with a `TextView` and three `Buttons` (Red, Green, Blue).
2. Link the UI elements in `MainActivity.java` using `findViewById`.
3. Set `OnClickListener`s on the buttons to change the `BackgroundColor` and `TextSize`.

Program (Java):

```
TextView tv = findViewById(R.id.textView);
Button btnRed = findViewById(R.id.btnRed);

btnRed.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        tv.setBackgroundColor(Color.RED);
        tv.setTextSize(24);
    }
});
```

```
    }  
});  
// Repeat similar logic for Green and Blue buttons
```

Output:

Clicking the "Red" button changes the TextView's background to Red and enlarges the text.

Result: Dynamic UI modification using GUI components executed successfully.

Ex 2 & 7: Layout Managers (Login Screen)

Aim: To design a Login Screen using `LinearLayout` where the username/password fields are vertical, and the Submit/Reset buttons are placed side-by-side using `layout_weight`.

Algorithm:

1. Set the root layout to `LinearLayout` with `orientation="vertical"`.
2. Add two `EditText` fields for Username and Password.
3. Add a nested `LinearLayout` with `orientation="horizontal"`.
4. Place two `Button`s inside the nested layout and assign `layout_weight="1"` to both.

Program (XML UI Design):

```
<LinearLayout android:layout_width="match_parent"  
    android:layout_height="wrap_content" android:orientation="horizontal">  
  
    <Button  
        android:id="@+id/btnSubmit"  
        android:layout_width="0dp"  
        android:layout_height="wrap_content"  
        android:layout_weight="1"  
        android:text="Submit" />  
  
    <Button  
        android:id="@+id/btnReset"  
        android:layout_width="0dp"  
        android:layout_height="wrap_content"  
        android:layout_weight="1"  
        android:text="Reset" />  
  
</LinearLayout>
```

Output:

The Submit and Reset buttons share the screen width equally (50/50 split) below the input fields.

Result: The Login screen using Layout Managers was designed successfully.

Ex 3: Calculator Interface (GridLayout)

Aim: To implement a Calculator interface using `GridLayout` and handle click events for digits 0-9 using a single `OnClickListener`.

Algorithm:

1. Use `GridLayout` to arrange digit buttons in a numeric pad format.
2. Create a single `View.OnClickListener` instance in Java.
3. Attach this single listener to all digit buttons, identifying the clicked button by casting the `View` to a `Button`.

Program (Java):

```
TextView display = findViewById(R.id.display);

View.OnClickListener digitListener = new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        Button b = (Button) v;
        display.append(b.getText().toString());
    }
};

findViewById(R.id.btn0).setOnClickListener(digitListener);
findViewById(R.id.btn1).setOnClickListener(digitListener);
// Continue for digits 2 through 9
```

Output:

Clicking '1', '4', and '7' appends the digits to the display screen showing "147".

Result: Calculator interface with unified click handling implemented successfully.

Ex 4 & 8: Graphical Primitives (Draw a House)

Aim: To write a program that draws a custom "House" shape on the screen using the `Canvas` and `Paint` classes.

Algorithm:

1. Create a custom class that extends `View`.
2. Override the `onDraw(Canvas canvas)` method.
3. Use `drawRect` for the house body and `drawLine` (or `drawPath`) for the roof.

Program (Java):

```
class HouseView extends View {
    Paint paint = new Paint();

    public HouseView(Context context) { super(context); }

    @Override
    protected void onDraw(Canvas canvas) {
        super.onDraw(canvas);

        // Draw House Body
        paint.setColor(Color.BLUE);
        canvas.drawRect(100, 300, 400, 600, paint);

        // Draw Roof
        paint.setColor(Color.RED);
        paint.setStrokeWidth(10);
        canvas.drawLine(100, 300, 250, 150, paint); // Left slant
        canvas.drawLine(250, 150, 400, 300, paint); // Right slant
    }
}
```

Output:

A graphical blue square base with a red triangular roof renders on the screen.

Result: Basic graphical primitives were successfully drawn on the Canvas.

Ex 5: Event Listening (Moving Circle)

Aim: To develop an app where a circle moves across the screen based on user touch coordinates.

Algorithm:

1. Create a custom `View` class.
2. Override `onTouchEvent` to get `event.getX()` and `event.getY()` .
3. Call `invalidate()` to force a redraw.
4. Draw the circle at the updated X, Y coordinates in `onDraw()` .

Program (Java):

```
class TouchCircleView extends View {
    float cx = 500, cy = 500;
    Paint paint = new Paint();

    public TouchCircleView(Context context) { super(context); }

    @Override
    public boolean onTouchEvent(MotionEvent event) {
        cx = event.getX();
        cy = event.getY();
        invalidate(); // Redraw screen
        return true;
    }

    @Override
    protected void onDraw(Canvas canvas) {
        paint.setColor(Color.GREEN);
        canvas.drawCircle(cx, cy, 100, paint);
    }
}
```

Output:

A green circle jumps to wherever the user taps or drags their finger on the screen.

Result: Touch event listening executed successfully.

Ex 9, 10, 13 & 16: Database CRUD Operations (SQLite)

Aim: To demonstrate CRUD operations by creating a "Student Records" app using `SQLite` .

Algorithm:

1. Create/Open an SQLite database using `openOrCreateDatabase` .
2. Execute `CREATE TABLE` query.
3. Use `execSQL` to **C**reate/**I**nsert and **D**ele~~te~~ data.

4. Use `rawQuery` and a `Cursor` to **Read** and display the data.

Program (Java):

```
SQLiteDatabase db = openOrCreateDatabase("StudentDB", MODE_PRIVATE, null);

// CREATE
db.execSQL("CREATE TABLE IF NOT EXISTS Employee(Name VARCHAR, Role VARCHAR);");

// INSERT
db.execSQL("INSERT INTO Employee VALUES('John Doe', 'Manager');");

// READ
Cursor cursor = db.rawQuery("SELECT * FROM Employee", null);
if(cursor.moveToFirst()) {
    String name = cursor.getString(0);
    String role = cursor.getString(1);
    System.out.println("Employee: " + name + " | Role: " + role);
}

// DELETE
db.execSQL("DELETE FROM Employee WHERE Name='John Doe';");
```

Output:

```
Employee: John Doe | Role: Manager
```

Result: SQLite database CRUD operations performed successfully.

Ex 11 & 12: Write Data to SD Card & Permission Check

Aim: To check if an SD Card is mounted with write permissions, and save a text file containing user input to the External Storage.

Algorithm:

1. Check `WRITE_EXTERNAL_STORAGE` permission.
2. Verify `Environment.getExternalStorageState()` equals `MEDIA_MOUNTED`.
3. Use `FileOutputStream` to write data to `getExternalFilesDir()`.

Program (Java):

```
String state = Environment.getExternalStorageState();
```

```

if (Environment.MEDIA_MOUNTED.equals(state)) {
    File file = new File(getExternalFilesDir(null), "UserInput.txt");
    try {
        FileOutputStream fos = new FileOutputStream(file);
        fos.write("Hello, SD Card!".getBytes());
        fos.close();
        Toast.makeText(this, "File Saved!", Toast.LENGTH_SHORT).show();
    } catch (IOException e) {
        e.printStackTrace();
    }
} else {
    Toast.makeText(this, "SD Card not writable",
Toast.LENGTH_SHORT).show();
}

```

Output:

Toast appears saying "File Saved!". UserInput.txt is created in the Android data folder.

Result: SD Card mounting validation and file writing completed successfully.

Ex 14: Stopwatch using Multi-threading

Aim: To build a "Stopwatch" application using Multi-threading to update the timer UI every second without freezing the main thread.

Algorithm:

1. Create an integer variable `seconds` .
2. Start a new background `Thread` .
3. Use `Thread.sleep(1000)` inside a `while(true)` loop.
4. Use `runOnUiThread` to safely update the `TextView`.

Program (Java):

```

TextView timerText = findViewById(R.id.timerText);
final int[] seconds = {0};

new Thread(new Runnable() {
    @Override
    public void run() {
        while (true) {
            try {
                Thread.sleep(1000);

```

```

        seconds[0]++;

        runOnUiThread(new Runnable() {
            @Override
            public void run() {
                timerText.setText("Time: " + seconds[0] + "s");
            }
        });
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
}
}).start();

```

Output:

The screen displays "Time: 1s", "Time: 2s", incrementing every second smoothly.

Result: Multi-threading utilized successfully for the Stopwatch application.

Ex 15 & 17: Background Service & Notification Manager

Aim: To create a background thread that simulates a long-running download and triggers a Notification in the status bar once complete.

Algorithm:

1. Start a background thread and use `Thread.sleep(5000)` to simulate a download.
2. Create a `NotificationChannel` (required for modern Android versions).
3. Use `NotificationCompat.Builder` to design the alert.
4. Trigger the alert using `NotificationManager`.

Program (Java):

```

new Thread(() -> {
    try {
        Thread.sleep(5000); // Simulate 5-second download
    } catch (Exception e) {}

    NotificationManager nm = (NotificationManager)
    getSystemService(NOTIFICATION_SERVICE);

    // Creating Notification

```



```
NotificationCompat.Builder builder = new
NotificationCompat.Builder(MainActivity.this, "CH1")
    .setSmallIcon(R.drawable.ic_launcher_foreground)
    .setContentTitle("Download Complete")
    .setContentText("Your file has been downloaded successfully.")
    .setPriority(NotificationCompat.PRIORITY_HIGH);

nm.notify(1, builder.build());
}).start();
```

Output:

After 5 seconds, a push notification drops down from the status bar saying "Download Complete".

Result: Notification Manager and background execution executed successfully.
